

LMS - Zoom Meeting Integration

Requirements & Feasibility Document

Project: SAAE Digital Platform (TanStack Start + Supabase / Lovable Cloud)

Date: September 24, 2026 | Version: 1.0 - Planning Only | Status: DRAFT

Audience: Product Owners, Admins, Instructors, Developers, Operators

This document explains WHY Zoom can be integrated with the current LMS, WHAT is required, and HOW delivery can be phased. No code was changed. No secrets are included. It is based on the current implementation: learning-management-system routes, server functions (createServerFn), Bunny Stream video pattern, role-based access, enrollment model, and AMS sessions.

Contents

1. Executive Summary
 2. Objectives & Use Cases
 3. Why Integration Is Feasible With the Current LMS
 4. Integration Options Compared
 5. Functional Requirements
 6. Technical Requirements (Account, Database, Backend, Webhooks, Frontend)
 7. Security, Privacy & Compliance
 8. UX & Localization Requirements
 9. Notifications, Calendar & Recordings
 10. Phased Delivery Plan & Estimates
 11. Risks, Assumptions & Mitigations
 12. Cost & Licensing Notes
 13. Open Decisions for Stakeholders
- Appendix A - Current LMS Reference Map

1. Executive Summary

Yes - Zoom Meeting can be integrated with the SAAE LMS. The LMS already has the exact extension points needed: a course / section / lesson hierarchy, online vs. onsite delivery modes, authenticated server functions, role-based access (lms_student, lms_instructor, lms_admin), enrollment-gated content, a proven secure-video pattern (Bunny Stream: create -> upload -> webhook -> signed playback), and an attendance system (AMS) for sessions.

A Zoom integration follows the same proven pattern: instructor creates a Live Session server-side -> Zoom API returns meeting IDs and join/start URLs -> students see a Join button only if enrolled -> webhooks record attendance and recordings -> recordings optionally feed back into Bunny as VOD. A minimal manual-link MVP can ship in days; a full API + webhook + embedded SDK integration is a 3-6 week track depending on scope.

2. Objectives & Use Cases

Primary objectives

- Add scheduled live classes to online (and hybrid onsite) courses without leaving the LMS.
- Give instructors one-click creation of Zoom meetings from inside the course editor.
- Give enrolled students a single trusted Join flow with correct role (host vs. participant).
- Capture attendance automatically for admin / certificates / AMS reporting.
- Reuse recordings as on-demand lessons via the existing Bunny pipeline.
- Send bilingual reminders and calendar invites to reduce no-shows.

Use cases covered

Use case	Description
Weekly live class	Recurring session linked to a course/section, visible in the course player.
Webinar / guest talk	One-off large meeting linked to a course or run standalone.
Office hours / 1-1	Instructor creates small meetings for enrolled students.
Hybrid onsite + Zoom	Onsite AMS session mirrored with a Zoom join link.
Catch-up VOD	Cloud recording becomes a Bunny lesson after the live event.

3. Why Integration Is Feasible With the Current LMS

This is not a rebuild. The current system already implements every prerequisite:

- Route hierarchy exists: learning-management-system.tsx (layout), catalog, courses/\$id (public detail), student/player/\$courseId (gated player), instructor/courses/\$id (editor), admin/* (ops). A Live Sessions card slots naturally into the player and editor.
- Server-function pattern proven: src/lib/*.functions.ts with createServerFn + requireSupabaseAuth middleware (see bunny-stream.functions.ts: createBunnyUpload, getBunnyPlayback, setLessonBunnyVideo, refreshBunnyLessonStatus). Zoom uses the same three calls: createLiveSession, getLiveJoin, refreshLiveStatus.
- Authorization model ready: user_roles table + has_lms_role / is_lms_admin helpers. Admin/instructor ownership

checks already used for lessons; same checks gate meeting creation and hide host start_url from students.

- Enrollment gating ready: lms_enrollments checked server-side before video playback. Same check gates Zoom join_url exposure - no client trust needed.
- Content model fits: lms_courses (with delivery_mode online/onsite), lms_sections, lms_lessons, lms_quizzes linked to ams_session_id. A new lms_live_sessions table links to course_id + optional section_id the same way quizzes do.
- Webhook precedent exists: /api/public/bunny-webhook with signature verification. /api/public/zoom-webhook reuses the same shape for meeting.started/ended, participant.joined/left, recording.completed.
- Email + i18n ready: Resend via connector gateway + React Email templates, bilingual ar/en strings in lms-i18n.ts, RTL/LTR layout already handled.
- Cloudflare Workers compatible: outbound fetch to Zoom REST API works from Workers; secrets stay server-only (never VITE_*).

4. Integration Options Compared

Option	Pros / Cons / Effort
A. Manual link (MVP)	PRO: 2-3 days, no Zoom API, instructor pastes Zoom/Meet/Jitsi URL. CON: no auto-attendance, no auto-recordings. Best first step.
B. Zoom API - scheduled (Recommended)	PRO: auto-create, role-based join/start URLs, webhooks for attendance + recordings. CON: needs Zoom Pro + OAuth app. 2-4 weeks.
C. Zoom Meeting SDK embedded	PRO: students join inside LMS player, full branding. CON: SDK weight, extra QA, stricter CSP. Add after B stabilizes (+1-2 weeks).
D. Google Meet / Jitsi / Teams	Meet: simple, cheap, weak attendance API. Jitsi: free/self-hosted, good for Syria sanctions/cost cases. Teams: only if corporate users demand it.

Recommendation for planning: ship A immediately to validate demand, then commit to B, then optionally C. Keep provider field generic (zoom | meet | jitsi | manual) so the data model never locks to Zoom.

5. Functional Requirements

5.1 Instructor

- Create / edit / delete a Live Session from instructor/courses/\$id: bilingual title, linked section (optional), starts_at + duration, timezone (default Asia/Damascus), capacity, auto-record flag, registration-required flag.
- View host start_url (never exposed to students), copy participant invite, test Join as host.
- View attendance per session (joined/left/duration) and recording status (processing/ready/failed).
- Convert a cloud recording to a Bunny VOD lesson with one click (Phase 3).

5.2 Student

- See Upcoming / Live Now / Ended + Recording states inside student/player/\$courseid and courses/\$id (teaser without link if not enrolled).
- Join button enabled only if: authenticated + actively enrolled + within join window (e.g. 15 min early) + session not cancelled.
- Countdown, timezone-correct time, Add-to-Calendar (.ics), join in browser or Zoom app.

- If missed: see recording when published (same enrollment check as Bunny playback).

5.3 Admin (lms_admin)

- List/filter all live sessions across courses, cancel any session, view attendance rollup, export XLSX (reuse admin-xlsx-export with formula sanitization).
- Manage Zoom connection health (token status, webhook log), retry failed recording imports.
- Link a live session to an AMS onsite session for hybrid attendance reconciliation.

6. Technical Requirements

6.1 Zoom account & app setup (ops, no code)

- Zoom Pro (or higher) account that owns the meetings; Meeting SDK needs a separate SDK key if Option C is chosen.
- Create Server-to-Server OAuth app in Zoom Marketplace -> yields ZOOM_ACCOUNT_ID, ZOOM_CLIENT_ID, ZOOM_CLIENT_SECRET (server-only).
- Create Webhook app / add event subscriptions: meeting.started, meeting.ended, participant joined/left, recording.completed -> yields ZOOM_WEBHOOK_SECRET_TOKEN.
- Configure public webhook URL: https://<domain>/api/public/zoom-webhook (must be reachable; test DNS/proxy like the existing lms.aisyria.org notes).
- Set recording storage (Zoom Cloud) + retention policy; decide auto-delete after import to Bunny.

6.2 Secrets contract (server-only, never VITE_*)

Secret	Purpose
ZOOM_ACCOUNT_ID	Server-to-Server OAuth identity.
ZOOM_CLIENT_ID / SECRET	Mint access tokens for Zoom REST API.
ZOOM_WEBHOOK_SECRET_TOKEN	Verify webhook signatures (HMAC).
ZOOM_SDK_KEY / SECRET (Phase 3)	Embedded Meeting SDK auth only.
CRON_SECRET (optional)	Protect reminder/cleanup scheduled jobs.

6.3 Data model (new tables, with RLS + grants before policies)

Proposed table public.lms_live_sessions:

id uuid PK, course_id uuid FK lms_courses, section_id uuid nullable FK lms_sections, title_ar/text, title_en/text, provider text (zoom|meet|jitsi|manual), meeting_id text, join_url text, start_url text (host-only), password text nullable, starts_at timestamptz, duration_min int, timezone text, status text (scheduled|live|ended|cancelled), record_flag bool, recording_url text nullable, recording_status text, ams_session_id uuid nullable, created_by uuid, created_at/updated_at.

Proposed table public.lms_live_attendance:

id uuid PK, session_id FK, user_id uuid, joined_at timestamptz, left_at timestamptz nullable, duration_sec int, source text (webhook|manual), unique(session_id, user_id, joined_at).

RLS: instructors (own courses) + lms_admin can write; enrolled students can read non-host columns only (hide start_url via a security-definer view or server function projection - never rely on client filtering).

6.4 Backend (follow bunny-stream.functions.ts pattern)

- createLiveSession (POST, auth): Zod-validate {courseId, sectionId?, titles, startsAt, duration, provider}; check instructor-owns-course or lms_admin; call Zoom POST /v2/users/me/meetings; persist row; return id + join_url.
- updateLiveSession / cancelLiveSession (POST, auth): same ownership; PATCH/DELETE on Zoom; update status.
- getLiveJoin (POST, auth): check enrollment or instructor/admin; enforce join window; return {joinUrl} for students, {joinUrl + startUrl} for host; log access in lms_audit_events.
- listCourseLiveSessions (GET/query): public teaser (times/titles/status) + gated URLs only via getLiveJoin.
- refreshLiveStatus (POST, auth instructor/admin): recovery path when webhook missed - GET Zoom meeting + update status (mirrors refreshBunnyLessonStatus).
- Zoom token cache: in-memory + DB fallback; refresh on 401; never expose tokens to browser.

6.5 Webhooks

- Route: src/routes/api/public/zoom-webhook.ts - verify HMAC signature with ZOOM_WEBHOOK_SECRET_TOKEN, return 401 on mismatch.
- Handle: meeting.started -> status=live; meeting.ended -> status=ended; participant.joined/left -> upsert lms_live_attendance; recording.completed -> recording_status=ready + store download URL (private) + enqueue Bunny import job.
- Idempotency: dedupe by Zoom event id; log to email/audit queue tables for replay.
- Guard: validate meeting_id maps to a known lms_live_sessions row before any write.

6.6 Frontend

- Instructor editor panel (in instructor/courses/\$id): form + validation (Zod + bilingual errors), session list with status pills, copy-invite, delete with confirm.
- Student player card (in student/player/\$courseId + courses/\$id teaser): Upcoming/Live/Ended states, countdown, Join button calling getLiveJoin, open in new tab or embedded SDK (Phase 3).
- Admin table (in admin/analytics or new admin/live-sessions): filters, attendance counts, recording state, export.
- State via TanStack Query (5-min stale, 30-min GC per router.tsx); no direct table writes from client - all through server functions.

7. Security, Privacy & Compliance

- Never trust the browser for enrollment, role, host URL, or attendance - all enforced server-side + RLS (same rule as quiz scores).
- Hide start_url, password, and webhook secrets from students; project only join_url through getLiveJoin.
- Verify Zoom webhook signatures; reject unknown meeting_ids; rate-limit public webhook (reuse chat rate-limit pattern: 15/min, 120/hr per IP).
- Passwords/waiting rooms on by default; do not log join URLs with passwords in plain application logs.
- Recordings are private by default; publish only to enrolled students; honor deletion requests (delete Zoom cloud copy after Bunny import if policy requires).
- PDPA-style: show recording notice pre-join in ar/en; store minimal attendance PII (user_id + timestamps only).

8. UX & Localization Requirements

- Full ar/en strings in lms-i18n.ts (titles, states, errors, emails); RTL/LTR verified on desktop + mobile.
- Times always rendered with explicit timezone (Asia/Damascus default) + user-local secondary line via date-fns.

- Accessible: Radix dialogs, focus management, keyboard-operable Join, reduced-motion respected, color-contrast safe status pills.
- Empty/error states: not-enrolled teaser, session cancelled, recording processing, webhook-delayed attendance (show 'syncing').

9. Notifications, Calendar & Recordings

- Transactional emails via existing Resend gateway + React Email templates: session scheduled, 24h + 1h reminder, recording ready, cancellation.
- Add-to-Calendar .ics download per session (server-generated, with correct TZ).
- Recording flow: Zoom Cloud -> webhook -> server downloads (with Zoom token) -> re-upload to Bunny via createBunnyUpload/TUS or Bunny direct pull -> setLessonBunnyVideo -> video_status processing->ready -> student sees VOD in player.
- Cron/scheduled job for reminders + stale-status sweeper (reuse scheduled-jobs.server.ts pattern); protect with CRON_SECRET.

10. Phased Delivery Plan & Estimates

Phase	Scope / Exit criteria
Phase 0 - Decisions (3-5 days)	Confirm use cases, Zoom licensing, timezone policy, attendance-needs-certificate rule, retention policy. Output: signed scope.
Phase 1 - Manual-link MVP (2-4 days)	DB tables + instructor URL field + gated Join button + basic emails. No Zoom API. Validates demand.
Phase 2 - Zoom API + webhooks (2-3 weeks)	OAuth, create/cancel, getLiveJoin, webhook attendance + status, admin view, integration tests. Production-ready live.
Phase 3 - Recordings to Bunny (1 week)	Recording import job, VOD publishing, retry UI.
Phase 4 - Embedded SDK + polish (1-2 weeks)	In-player Meeting SDK, calendar ICS, AMS hybrid reconciliation, analytics, e2e ar/en tests.

Estimates assume one full-stack dev + part-time QA + existing Supabase/Cloudflare access. Add 30% if Jitsi self-hosting or Teams support is required in parallel.

11. Risks, Assumptions & Mitigations

Risk / Assumption	Mitigation
Zoom availability / sanctions in Syria	Keep provider-agnostic model; support manual + Jitsi fallback; test join from target networks early.
Webhook delivery delay/failure	refreshLiveStatus recovery button + cron sweeper; never block Join on webhook state.
Token expiry / 401 from Zoom	Central token manager with refresh + retry once; alert admin on repeated failures.

Timezone confusion	Store timestamptz + explicit timezone; render dual times; e2e test DST edges.
Recording cost / storage blowup	Retention policy + auto-import-then-delete; quota dashboard for admins.
In-memory limits on Workers	No local rate-limit reliance for webhooks; use DB-backed idempotency.

12. Cost & Licensing Notes

- Zoom: at least 1 Pro license per concurrent host; Webinar add-on only if >300 attendees or panelist features needed; cloud-recording storage overage billed separately - confirm current Zoom pricing in 2026 before committing.
- Dev/infra: no new infra beyond existing Supabase + Cloudflare Workers + Resend + Bunny; Jitsi self-hosting (if chosen) adds a VPS + TURN server cost.
- Hidden effort: QA across Zoom app vs. browser, Arabic email templates, support runbook for instructors.

13. Open Decisions for Stakeholders

- 1. Is live mandatory for certificates, or supplementary? (drives attendance strictness)
- 2. Who may create meetings - any instructor or only admins? (drives licensing count)
- 3. Auto-record all sessions? How long to retain Zoom cloud copies after Bunny import?
- 4. Hybrid onsite + Zoom, or online-only for Phase 2?
- 5. Embedded in-player join (SDK) required at launch, or new-tab join acceptable?
- 6. Budget sign-off for Zoom Pro / Webinar / extra recording storage?

Appendix A. Current LMS Reference Map

- Layout: `src/routes/learning-management-system.tsx`; catalog: `learning-management-system.catalog.tsx`; detail: `learning-management-system.courses.$id.tsx`
- Player: `learning-management-system.student.player.$courseId.tsx`; quizzes: `student/quiz/$courseId.tsx`
- Instructor editor: `learning-management-system.instructor.courses.$id.tsx`; admin: `learning-management-system.admin/*`
- Video precedent: `src/lib/bunny-stream.functions.ts`; webhook: `src/routes/api/public/bunny-webhook.ts`
- Auth/roles: `src/lib/lms-auth.functions.ts`, `lms-roles.ts`, `lms-permissions.ts`; i18n: `lms-i18n.ts`
- Docs: `docs/PROJECT-DOCUMENTATION.md` (sections 6.2, 7.2-7.7, 9-10); migrations: `supabase/migrations/*.sql`

Prepared as planning input only. Next step after approval: technical design with exact Zod schemas, SQL migration draft, and Figma states for Upcoming / Live / Ended + Recording.